

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

LABORATORY REPORT



AGILE SOFTWARE DEVELOPMENT LAB

Lab No. 3

BESE III YEAR, VI-SEM

Submitted By:

Prabin Thapa

Submitted To:

Er. Rajendra Bahadur Thapa

Date: July 4, 2026

NAME OF EXPERIMENT: Implementing different types of Testing

AIM: To study and implement different software testing techniques and frameworks, including **test case generation, unit testing using JEST and JUnit, automation testing using Selenium, and load testing using Artillery**, in order to ensure software quality, correctness, and performance.

Objectives:

1. To understand the importance of software testing in software development.
2. To generate effective test cases for software applications.
3. To perform unit testing using the JEST framework.
4. To perform unit testing using the JUnit framework.
5. To automate web application testing using Selenium.
6. To evaluate application performance using Artillery load testing.
7. To analyze testing results and identify software defects.

THEORY:

Introduction to Software Testing

Software Testing is the process of verifying and validating that a software application functions according to its specified requirements and is free from defects. It helps improve software quality, reliability, security, and performance before deployment.

Testing can be performed manually or automatically using various testing frameworks and tools.

1. Test Case Generation

A test case is a set of conditions or inputs used to verify whether a software application behaves as expected.

A good test case includes:

1. Test Case ID
2. Objective
3. Input Data
4. Expected Output
5. Actual Output
6. Test Result (Pass/Fail)

2. JEST Framework

JEST is a JavaScript testing framework developed by Meta. It is primarily used for unit testing JavaScript and Node.js applications.

Features:

1. Simple syntax
2. Automatic test execution
3. Snapshot testing
4. Code coverage reports
5. Mocking support

3. JUnit

JUnit is a popular testing framework used for Java applications. It helps developers write and execute unit tests efficiently.

Features:

1. Test annotations
2. Assertions
3. Test reports
4. Automated execution
5. Integration with IDEs and build tools

JUnit simplifies testing of Java classes and methods.

4. Selenium

Selenium is an open-source automation testing tool used to test web applications by simulating user interactions.

It supports multiple browsers such as Chrome, Firefox, Edge, and Safari.

Common Selenium operations include:

1. Opening a webpage
2. Clicking buttons
3. Entering text
4. Selecting options
5. Verifying page content

5. Load Testing (Artillery)

Load Testing measures how an application performs under multiple concurrent users.

Artillery is a modern load-testing framework used for testing APIs and web applications.

It helps measure:

1. Response time
2. Throughput
3. Requests per second
4. Error rate

5. Concurrent users

Load testing identifies performance bottlenecks before deployment.

Algorithm

1. Develop the application to be tested.
2. Identify the functional requirements.
3. Generate suitable test cases.
4. Perform unit testing using JEST (JavaScript) or JUnit (Java).
5. Verify test results using assertions.
6. Automate browser testing using Selenium.
7. Execute load testing using Artillery.
8. Analyze the testing reports.
9. Fix detected defects if any.
10. Repeat testing until all tests pass.
11. End.

CODE:

Program (Python)

1. Test Case Generation

Program:

```
def add(a, b):  
    return a + b  
print(add(5, 3))
```

Test Cases:

Test Case	Input	Expected Output	Actual Output	Result
TC01	5, 3	8	8	Pass
TC02	-2, 5	3	3	Pass
TC03	0, 0	0	0	Pass

Output

8

2. JEST Framework (JavaScript)

Source File:

```
function add(a, b) {  
  return a + b;  
}  
module.exports = add;
```

Test File:

```
const add = require('./add');  
  
test("Adding positive numbers", () => {  
  expect(add(5,3)).toBe(8);  
});  
  
test("Adding zero", () => {  
  expect(add(0,0)).toBe(0);  
});  
  
test("Adding negative number", () => {  
  expect(add(-2,5)).toBe(3);  
});
```

Run Command

```
npm test
```

Output:

```
PASS add.test.js  
  
✓ Adding positive numbers  
✓ Adding zero  
✓ Adding negative number  
  
Test Suites: 1 passed  
Tests: 3 passed
```

3. JUnit (Java)

```
import static org.junit.Assert.assertEquals;
import org.junit.Test;

public class CalculatorTest {

    @Test
    public void testAddition() {
        assertEquals(8, 5 + 3);
    }

    @Test
    public void testZero() {
        assertEquals(0, 0 + 0);
    }
}
```

Output

JUnit Test Results

Tests Run: 2

Failures: 0

Status: PASSED

4. Selenium (Python)

Install Selenium:

```
pip install selenium
```

Program:

```
from selenium import webdriver

driver = webdriver.Chrome()

driver.get("https://www.google.com")

print(driver.title)

driver.quit()
```

Output:

```
Google
```

Browser opens automatically and closes after printing the page title.

5. Load Testing (Artillery)

Install:

```
npm install -g artillery
```

artillery.yml :

```
config:
  target: "https://jsonplaceholder.typicode.com"

phases:
  - duration: 30
    arrivalRate: 5

scenarios:
  - flow:
    - get:
      url: "/posts"
```

Run:

```
artillery run artillery.yml
```

Sample Output

Started phase

Requests completed: 150

Response time:

Min: 20 ms

Max: 140 ms

Average: 45 ms

Errors: 0

Completed successfully

RESULT

The implementation of different software testing techniques was successfully carried out. Test cases were generated and executed, unit testing was performed using JEST and JUnit, browser automation was demonstrated using Selenium, and application performance was evaluated using Artillery load testing.

CONCLUSION

This experiment provided practical knowledge of different software testing methodologies and frameworks used in modern software development. Test case generation ensured that functional requirements were validated systematically, while JEST and JUnit demonstrated the effectiveness of automated unit testing. Selenium enabled automated browser testing by simulating real user interactions, and Artillery was used to assess application performance under load. Collectively, these testing techniques help improve software quality, reliability, functionality, and scalability, making them essential components of the software development lifecycle.